JAVA PROGRAMMING / DSA

Chapter 38: Bit Manipulation

Personal Notes

Every number in a computer is really a sequence of BITS — 0s and 1s. Most of the time you don't think about that, but a whole family of interview problems becomes shockingly elegant when you work at the bit level: finding a unique number in an array, counting set bits, generating all subsets, checking powers of two, swapping without a temp variable.

Bit manipulation is a small topic but delivers massive returns. XOR tricks alone solve half a dozen classic problems in O(n) time and O(1) space. And bitmask techniques give you a whole new way to represent state in DP and backtracking problems. This chapter covers the operators, the patterns, and the interview classics.

1. Binary Representation Refresher

Every integer is stored in binary — a sequence of bits, each 0 or 1. In Java, int is 32 bits. The rightmost bit is bit 0 (the "least significant"), and the leftmost is bit 31.

```
Decimal:   13
Binary:    0000 0000 0000 0000 0000 0000 0000 1101
           ↑                                  ↑↑↑↑
           bit 31                        bit 3 2 1 0

13 = 1×2³ + 1×2² + 0×2¹ + 1×2⁰ = 8 + 4 + 0 + 1 = 13
```

Common Binary Values to Recognize

Decimal	Binary (8-bit view)	Note
1	`0000 0001`	2^0
2	`0000 0010`	2^1
4	`0000 0100`	2^2
8	`0000 1000`	2^3
16	`0001 0000`	2^4
255	`1111 1111`	$2^8 - 1$
-1	`1111...1111`	All bits set (two's complement)

Negative numbers use TWO'S COMPLEMENT: -1 is stored as ALL bits set to 1. This is why negative numbers appear as huge unsigned values when you print them in binary — but the arithmetic works out cleanly because addition and subtraction give the right results without extra hardware.

2. The Six Bitwise Operators

Op	Symbol	Effect	Example
AND	**&**	1 only if BOTH bits are 1	`1100 & 1010 = 1000`
OR	**|**	1 if EITHER bit is 1	`1100 | 1010 = 1110`
XOR	**^**	1 if bits DIFFER	`1100 ^ 1010 = 0110`
NOT	**~**	Flip every bit	`~1100 = ...0011 (32-bit)`
L-shift	**<<**	Shift bits LEFT (multiply by 2)	`5 << 2 = 20`
R-shift	**>>**	Shift bits RIGHT (divide by 2)	`20 >> 2 = 5`

Visual: AND, OR, XOR on 12 and 10

```
     12 = 1100
     10 = 1010

     12 & 10 = 1000   = 8     (only where BOTH are 1)
     12 | 10 = 1110   = 14    (where EITHER is 1)
     12 ^ 10 = 0110   = 6     (where they DIFFER)
     ~12     = 11...0011 = -13  (all bits flipped)
```

Shift Operators

```
5  << 1 = 10    // 0101 → 1010    (multiply by 2)
5  << 3 = 40    // 0101 → 101000  (multiply by 8)
20 >> 2 = 5     // 10100 → 101   (divide by 4, floor)

// Java also has >>> (unsigned right shift)
// For negative numbers:
-8 >> 1  = -4        // preserves sign bit
-8 >>> 1 = 2147483644  // fills with 0s (unsigned)
```

3. Essential Bit Manipulation Tricks

These are the foundational building blocks. Every bit-manipulation problem uses combinations of them.

3.1 Check if the i-th Bit is Set

```
boolean isSet(int n, int i) {
    return (n & (1 << i)) != 0;
}

// Example: n = 13 (1101), i = 2
// 1 << 2 = 0100
// 1101 & 0100 = 0100 (non-zero) → true
// bit 2 of 13 IS set
```

3.2 Set the i-th Bit

```
int setBit(int n, int i) {
    return n | (1 << i);
}

// n = 8 (1000), i = 1
// 1 << 1 = 0010
// 1000 | 0010 = 1010 → 10
```

3.3 Clear the i-th Bit

```
int clearBit(int n, int i) {
    return n & ~(1 << i);
}

// n = 15 (1111), i = 2
// 1 << 2 = 0100 → ~0100 = ...1011
// 1111 & ...1011 = 1011 → 11
```

3.4 Toggle the i-th Bit

```
int toggleBit(int n, int i) {
    return n ^ (1 << i);
}

// XOR with 1 flips a bit. XOR with 0 leaves it alone.
```

3.5 Get the Lowest Set Bit

```
int lowestBit(int n) {
    return n & -n;
}

// n = 12 (1100)
// -n = ...10100  (two's complement)
// 12 & -12 = 0100 = 4 (the lowest set bit)
```

This is one of the most useful tricks in the toolkit. It isolates the RIGHTMOST 1 bit — used heavily in Fenwick trees, bit iteration, and subset enumeration.

3.6 Remove the Lowest Set Bit

```
int removeLowest(int n) {
    return n & (n - 1);
}

// n = 12 (1100)
// n - 1 = 1011
// 1100 & 1011 = 1000 → 8
```

Combined with a loop, this counts set bits in O(popcount) — one iteration per set bit.

3.7 Check if a Number is a Power of 2

```
boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

// A power of 2 has exactly ONE bit set (like 1000, 10000, etc).
// Removing its lowest set bit gives 0.
// 8 = 1000, 8-1 = 0111, 8 & 7 = 0 → power of 2 ✓
// 6 = 0110, 6-1 = 0101, 6 & 5 = 0100 → not power of 2
```

4. The XOR Superpower

XOR is the most magical bitwise operator for interview problems. Learn these four properties by heart:

```
    1. Self-cancellation:    x ^ x = 0
    2. Identity:              x ^ 0 = x
    3. Commutative:           a ^ b = b ^ a
    4. Associative:           (a ^ b) ^ c = a ^ (b ^ c)

    Consequence: XOR-ing the same number twice cancels it out.
                 Order doesn't matter — you can XOR everything in any
sequence.
```

Why This is Powerful

If you XOR every element in an array where every value appears an even number of times, the result is 0. If one value appears an odd number of times (say once), everything else cancels — and you're left with just that value.

> **The XOR trick in one sentence:** "XOR is a pairing operator — matched pairs disappear, unmatched values remain." This single insight solves Single Number, Missing Number, Find the Duplicate, and several other classic problems in O(n) time and O(1) space.

5. Counting Set Bits

Also called "Hamming Weight" or "popcount." Three ways to do it — pick based on the situation.

Method 1: Loop and Check Each Bit

```
int countBits(int n) {
    int count = 0;
    while (n != 0) {
        if ((n & 1) == 1) count++;
        n >>>= 1;
    }
    return count;
}

// O(32) — always processes 32 bits regardless of the value.
```

Method 2: Brian Kernighan's Trick (Faster)

```
int countBits(int n) {
    int count = 0;
    while (n != 0) {
        n = n & (n - 1);   // remove lowest set bit
        count++;
    }
    return count;
}

// O(popcount) — one iteration per set bit.
// For sparse numbers this is much faster.
```

Method 3: Java Built-in (Interview Bonus)

```
int count = Integer.bitCount(n);   // fastest — uses hardware instruction
```

> **Use >>> not >> for right shift:** For negative numbers, >> preserves the sign bit — so 1s keep flowing in from the left. That means your loop never terminates! Always use >>> (unsigned right shift) when scanning bits, or work only with non-negative values.

6. Classic Problem 1 — Single Number

Given an array where every element appears twice except for one, find that one. Constraint: O(n) time, O(1) space (no HashMap allowed).

The XOR Approach

```
public int singleNumber(int[] nums) {
    int result = 0;
    for (int n : nums) result ^= n;
```

```
    return result;
}

// XOR all numbers. Every pair cancels (x^x=0).
// Only the lone number remains.
// Time: O(n)  Space: O(1)
```

Dry Run for [4, 1, 2, 1, 2]

```
result = 0
result ^= 4  →  4
result ^= 1  →  5
result ^= 2  →  7
result ^= 1  →  6
result ^= 2  →  4

Answer: 4
```

7. Classic Problem 2 — Missing Number

Given an array of n distinct numbers taken from 0 to n, find the missing one. Two elegant approaches — the XOR one avoids overflow risks.

Approach 1: XOR

```
public int missingNumber(int[] nums) {
    int result = nums.length;   // start with n (the expected max)
    for (int i = 0; i < nums.length; i++) {
        result ^= i ^ nums[i];
    }
    return result;
}

// XOR every index with every value.
// Missing number appears in indices but not values (or vice versa).
// Everything else cancels.
```

Approach 2: Sum Difference

```
public int missingNumber(int[] nums) {
    int n = nums.length;
    int expected = n * (n + 1) / 2;
    int actual = 0;
    for (int num : nums) actual += num;
    return expected - actual;
}
```

Both work. XOR is safer for huge n (no overflow); sum is more intuitive.

8. Classic Problem 3 — Number of 1 Bits (Hamming Weight)

Return how many 1 bits are set in the binary representation of an integer.

```
public int hammingWeight(int n) {
    int count = 0;
    while (n != 0) {
        n &= (n - 1);
        count++;
    }
    return count;
}

// Brian Kernighan's method — one loop iteration per set bit.
// Example: 11 = 1011 → 3 set bits
```

9. Classic Problem 4 — Reverse Bits

Reverse the bit order of a 32-bit integer. So 0000...0001 becomes 1000...0000.

```
public int reverseBits(int n) {
    int result = 0;
    for (int i = 0; i < 32; i++) {
        result <<= 1;              // make room for next bit
        result |= (n & 1);          // grab the lowest bit of n
        n >>>= 1;                    // shift n right
    }
    return result;
}

// Time: O(32) = O(1)  Space: O(1)
```

10. Classic Problem 5 — Power of Two

```
public boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}

// Any power of 2 has EXACTLY one bit set:
//   1  = 0001
//   2  = 0010
//   4  = 0100
//   8  = 1000
// Removing the lowest set bit gives 0 for all of them.
```

The n > 0 check matters: For n = 0, (0 & -1) == 0, so it would falsely report 0 as a power of 2. Always guard with n > 0. Same trap for isPowerOfFour and similar variants.

11. Classic Problem 6 — Sum of Two Integers Without + or -

A famous interview question — compute a + b using only bitwise operators. The key insight: addition = XOR (sum without carry) + shifted AND (the carry).

```
public int getSum(int a, int b) {
    while (b != 0) {
        int carry = (a & b) << 1;   // bits where BOTH have 1 → carry left
        a = a ^ b;                    // XOR = sum without carry
        b = carry;                    // apply carry next iteration
    }
    return a;
}

// Example: 3 + 5
//   a=0011, b=0101
//   carry = (0011 & 0101) << 1 = 0001 << 1 = 0010
//   a = 0011 ^ 0101 = 0110
//   b = 0010
//
//   a=0110, b=0010
//   carry = (0110 & 0010) << 1 = 0010 << 1 = 0100
//   a = 0110 ^ 0010 = 0100
//   b = 0100
//
//   a=0100, b=0100
//   carry = 0100 << 1 = 1000
//   a = 0100 ^ 0100 = 0000
//   b = 1000
//
//   a=0000, b=1000
//   carry = 0
//   a = 1000
//   b = 0 → return 8 ✓
```

12. Bitmask — Representing Subsets

Every bit in an integer can represent "is item i included?" This turns a set of choices into a single integer, enabling extremely fast subset operations.

```
    Items:      [A, B, C, D]     (positions 0, 1, 2, 3)

    Bitmask     Set represented
    ─────────   ──────────────────
    0000  = 0   {}
    0001  = 1   {A}
    0010  = 2   {B}
    0011  = 3   {A, B}
```

```
     0101  = 5   {A, C}
     1111  = 15  {A, B, C, D}

     n items → 2ⁿ possible subsets, each an integer from 0 to 2ⁿ-1
```

Generate All Subsets Using Bitmasks

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    int n = nums.length;
    int total = 1 << n;   // 2^n

    for (int mask = 0; mask < total; mask++) {
        List<Integer> subset = new ArrayList<>();
        for (int i = 0; i < n; i++) {
            if ((mask & (1 << i)) != 0) subset.add(nums[i]);
        }
        result.add(subset);
    }
    return result;
}

// For nums = [1, 2, 3], generates:
//   mask 000 = []
//   mask 001 = [1]
//   mask 010 = [2]
//   mask 011 = [1, 2]
//   mask 100 = [3]
//   mask 101 = [1, 3]
//   mask 110 = [2, 3]
//   mask 111 = [1, 2, 3]
```

Bitmask vs backtracking: For small n (up to ~20), bitmask enumeration is often cleaner than backtracking recursion. Each subset is JUST an integer. No stack, no recursion, just a loop. For larger n, the 2^n cost dominates and you need better algorithms.

13. Bitmask DP — A Preview

Some DP problems have state = "which subset of items have I used?" Storing that as an int lets you fit exponentially large state into a simple array indexed by mask.

Example: Traveling Salesman Skeleton

```
// dp[mask][i] = minimum cost to visit cities in 'mask', ending at city i
int[][] dp = new int[1 << n][n];

// mask = 000...1 means only city 0 visited
// mask = 111...1 means all cities visited
```

```
for (int mask = 0; mask < (1 << n); mask++) {
    for (int i = 0; i < n; i++) {
        if ((mask & (1 << i)) == 0) continue;   // i not in mask
        // ... try coming to i from every previous city j in mask
    }
}
```

Bitmask DP applies whenever "which items are chosen" is small enough (say n ≤ 20). It's how TSP, Assign-Tasks, and Shortest-Superstring problems are typically solved.

14. Useful Java Bit Utilities

Method	Purpose
`Integer.bitCount(n)`	Count of set bits (popcount)
`Integer.toBinaryString(n)`	String of the binary representation
`Integer.numberOfLeadingZeros(n)`	Count of leading 0 bits
`Integer.numberOfTrailingZeros(n)`	Count of trailing 0 bits
`Integer.highestOneBit(n)`	Isolate the HIGHEST set bit
`Integer.lowestOneBit(n)`	Isolate the LOWEST set bit (same as n & -n)
`Integer.reverse(n)`	Reverse the 32-bit representation
`Integer.parseInt("1101", 2)`	Parse a binary string to int
`Long.bitCount(long)`	Same as bitCount but for 64-bit longs

For most interviews, writing the manual versions is a good demonstration of understanding — but knowing the built-ins is handy for follow-up questions and cleaner code.

15. Common Mistakes

Mistake 1: Using >> for negative numbers

```
int n = -8;
n >> 1;    // -4 — sign bit is preserved, may loop forever
n >>> 1;   // 2147483644 — logical shift, fills with 0

// If you're iterating through bits, ALWAYS use >>>
```

Mistake 2: Operator precedence surprises

```
if (n & 1 == 0) ...    // ✗ WRONG — parses as n & (1 == 0) = n & 0 = 0
if ((n & 1) == 0) ...   // ✓ correct
```

Mistake 3: Confusing OR-assignment with plus

```
mask |= (1 << i);   // sets bit i
mask +  (1 << i);   // ✗ this ADDS — different when bit i was already set
```

Mistake 4: Forgetting the n > 0 guard for power-of-two check

`(n & (n - 1)) == 0` is also true for n = 0. Always guard with n > 0.

Mistake 5: Shift by 32 or more

```
int x = 1 << 32;   // does NOT give 0 — Java uses only the low 5 bits of
the shift amount
                    // So 1 << 32 = 1 << 0 = 1 (surprising!)
long y = 1L << 32;  // ✓ works — long is 64 bits
```

Mistake 6: Assuming char/byte are unsigned

In Java, byte is signed (-128 to 127) but char is unsigned (0 to 65535). Bit operations on byte often surprise beginners because of sign extension when converting to int.

Mistake 7: Not initializing the accumulator to 0 for XOR

```
int result = 0;                    // ✓ correct — 0 is XOR's identity
int result = nums[0];              // OK too — start with the first
int result = Integer.MAX_VALUE;    // ✗ WRONG — non-zero start breaks it
```

16. When to Reach for Bit Manipulation

If you need to…	Use
Find the ONE non-repeating number (others repeat)	**XOR everything → the loner remains**
Find a MISSING number in 0..n	**XOR expected vs actual**
Generate all subsets of a small set (n ≤ 20)	**Bitmask enumeration**
Track visited items in a fixed small set	**Bitmask instead of boolean[]**
Count how many 1 bits (Hamming weight)	**Brian Kernighan or bitCount**
Check power of 2 / power of 4	**n & (n-1) == 0 trick**
Swap without a temp variable	**a ^= b; b ^= a; a ^= b;**
State compression in DP (TSP, assignments)	**Bitmask DP**

If you need to...	Use
Encoding permissions / flags	**OR bits into a single int**

17. Best Practices

- Always use >>> (unsigned right shift) when iterating bits — >> traps you on negatives
- Wrap bitwise expressions in parentheses — operator precedence surprises everyone
- For power-of-2 checks, always guard with n > 0
- Prefer Integer.bitCount() and other built-ins in production code — they use hardware instructions
- For subset generation with n ≤ 20, bitmask is cleaner than backtracking
- For n > 30, use long instead of int to avoid overflow in shifts
- Memorize the four XOR properties — they unlock a family of problems
- When debugging, print Integer.toBinaryString(n) to see the actual bits

18. Quick Reference

Concept	Meaning / Trick
&	AND — 1 only if both bits are 1
|	OR — 1 if either bit is 1
^	XOR — 1 if bits differ
~	NOT — flip every bit
<<	Left shift — multiply by 2^k
>>	Right shift (arithmetic — preserves sign)
>>>	Right shift (logical — fills with 0)
`Check bit i`	(n & (1 << i)) != 0
`Set bit i`	n |= (1 << i)
`Clear bit i`	n &= ~(1 << i)
`Toggle bit i`	n ^= (1 << i)
`Lowest set bit`	n & -n
`Remove lowest set bit`	n & (n - 1)

Concept	Meaning / Trick
`Power of 2?`	n > 0 && (n & (n-1)) == 0
`Count set bits`	Integer.bitCount(n) or Brian Kernighan
`Subsets of n items`	0 to (1 << n) - 1

19. Practice Problems

Practice in order — first the foundational patterns, then classics, then bitmask techniques.

Foundational

1. Check if a specific bit is set in an integer.
2. Set, clear, and toggle a bit at position i.
3. Swap two integers WITHOUT a temp variable using XOR.
4. Count the number of 1 bits (Number of 1 Bits, LeetCode 191).
5. Reverse the bits of a 32-bit integer (LeetCode 190).
6. Check if a number is a power of 2 (LeetCode 231).
7. Check if a number is a power of 4 (LeetCode 342).

XOR Classics

8. Single Number (LeetCode 136) — every other appears twice.
9. Missing Number (LeetCode 268).
10. Find the Duplicate Number (LeetCode 287) — the XOR variant.
11. Single Number II (LeetCode 137) — others appear THREE times.
12. Single Number III (LeetCode 260) — TWO singles, others appear twice.

Sum & Arithmetic

13. Sum of Two Integers without + or - (LeetCode 371).
14. Divide Two Integers using only bit shifts (LeetCode 29).
15. Counting Bits (LeetCode 338) — return bit counts for 0 to n in one pass.

Bitmask & Subsets

16. Generate All Subsets using bitmask iteration (LeetCode 78).
17. Count Subsets Sum Equal to Target using bitmask enumeration for small n.
18. Maximum XOR of Two Numbers in an Array (LeetCode 421) — with Trie or greedy).
19. Total Hamming Distance between all pairs (LeetCode 477) — bit-by-bit contribution.

Advanced Bitmask DP

20. Shortest Path Visiting All Nodes (LeetCode 847) — bitmask + BFS.
21. Traveling Salesman Problem — mask DP starter.
22. Partition to K Equal Sum Subsets (LeetCode 698) — bitmask + DFS.
23. Can I Win (LeetCode 464) — bitmask + memoization.

20. Final Takeaway

Bit manipulation looks intimidating at first — all those symbols and shifts — but it collapses into a small set of patterns once you see them a few times. The XOR self-cancellation trick alone will make you seem magical in an interview: an O(n) time, O(1) space solution to what looks like a HashMap problem.

For interview prep, master the six operators, the seven core tricks (isSet, setBit, clearBit, toggleBit, lowestBit, removeLowest, power-of-2 check), and the four XOR properties. Those combined with bitmask enumeration handle probably 90% of bit manipulation questions you'll ever see.

> **Key Takeaway:** Bit manipulation runs at hardware speed with zero extra memory. XOR is the pairing operator (matched pairs cancel). n & -n isolates the lowest set bit. n & (n-1) removes it. (n & (n-1)) == 0 checks power of 2. For n ≤ 20 subsets, iterate mask from 0 to 2^n-1 and check bits. Memorize these building blocks and a whole category of problems becomes routine.

21. What's Next?

With bit manipulation covered, the DSA toolkit is nearly complete. Remaining specialized topics:

- Monotonic Stack / Deque — next greater element, largest rectangle in histogram
- Two Pointers in depth — palindrome, 3-sum, container with most water
- Prefix Sums — range queries, subarrays with negatives
- Segment Tree / Fenwick Tree — range queries with updates
- Advanced Strings — KMP, Rabin-Karp, suffix arrays
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points
- Greedy Algorithms — interval scheduling, activity selection
- System Design — for later interview rounds